

Eulerian Tour Algorithms: Mail Delivery Problem

Group Members: Denis, Hugo, Melvyn, Mikal

Instructor: Ilham Gurat Adilion

Date: September 23, 2026

Introduction & Problem Overview

This report explores practical applications of Eulerian trail algorithms to optimize route planning for postal delivery services. The primary objective is to evaluate different algorithmic approaches for finding an Eulerian circuit within a city's road network.

Problem Context

Consider a software engineering scenario for a city's postal service: a mail carrier must depart from the post office, traverse every street segment exactly once to maximize efficiency, and ultimately return to the starting location.

This routing challenge can be modeled as a graph theory problem where street intersections represent vertices (nodes) and road segments represent edges.

The benchmark implementation and test cases are based on the CSES Mail Delivery problem: <https://cses.fi/problemset/task/1691>.

A. Step by step executions on sample graph

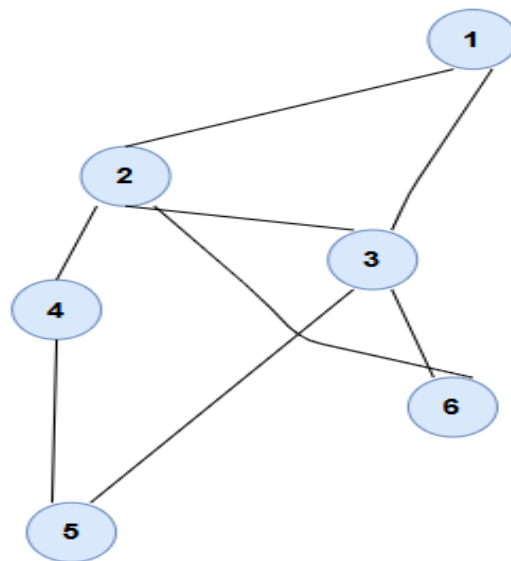
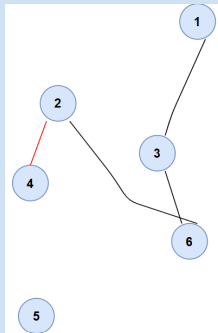
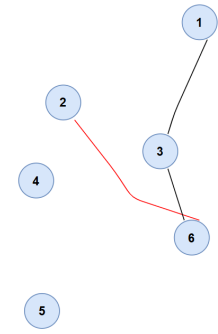
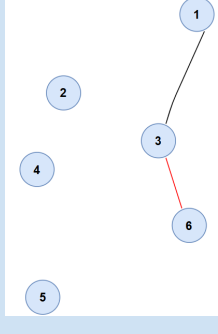
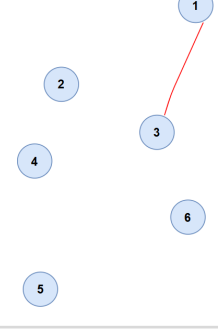


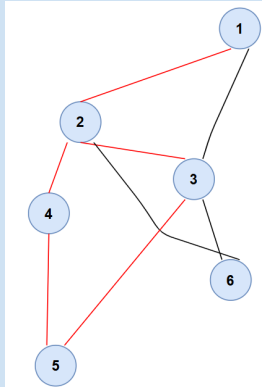
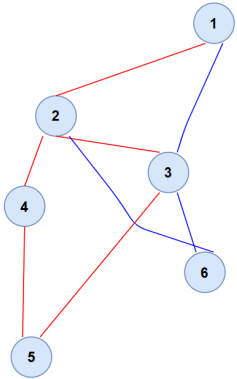
Fig 1 : Graph sample used for the walkthrough

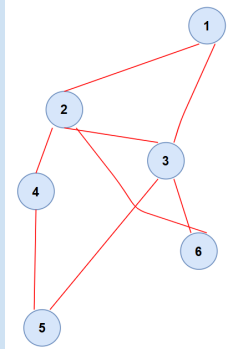
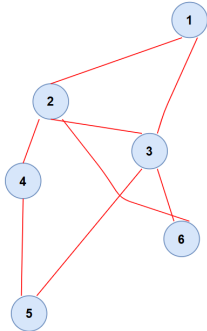
Fleury's Algorithm Execution

Step	Edges candidate	Edge traversed	Choice	Graph
1	(1,2), (1,3)	(1,2)	At start node 1, all degrees even, no bridges exist. Pick (1, 2).	
2	(2,3), (2,4), (2,6)	(2,3)	(2, 3) tested: path 2-4-5-3 still connects 2 and 3 in G.	
3	(3,1), (3,5), (3,6)	(3,5)	(3, 1) is a bridge. Removing (3, 1) strands node 1 before visiting 4, 5, 6. Must NOT take (3, 1). Pick non-bridge (3, 5).	
4	(5,4)	(5,4)	Only 1 edge remaining incident to 5. Forced choice (no alternative).	

5	(4,2)	(4,2)	Only 1 edge remaining incident to 4. Forced choice.	
6	(2,6)	(2,6)	Only 1 edge remaining incident to 2. Forced choice.	
7	(6,3)	(6,3)	Only 1 edge remaining incident to 6. Forced choice.	
8	(3,1)	(3,1)	Only 1 edge remaining incident to 3. Forced choice, closes circuit at 1.	

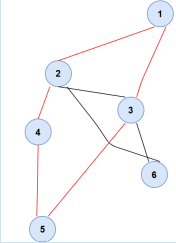
Hierholzer's Algorithm Execution

Step	Current Path / Stack	Edge Traversed	Action
1	[1, 2, 3, 5, 4, 2]	$(1,2) \rightarrow (2,3) \rightarrow (3,5) \rightarrow (5,4) \rightarrow (4,2)$	Initial Main Walk: Start at node 1 and traverse unvisited edges until returning to node 2 (which still has unused incident edges like $(2,6)$). 
2	[1, 2, 6, 3, 1, 2]	$(2,6) \rightarrow (6,3) \rightarrow (3,1)$	Secondary Exploration: From node 2, take the unused edge to node 6, continue to node 3, and loop back to node 1 to form a secondary cycle. 

3	[1, 2, 3, 5, 4, 2, (6, 3, 1)]	Integration	Sub-tour Splicing: Insert the secondary cycle into the main path at node 2 where the fresh exploration branched off. 
4	Final Circuit: 1, 2, 3, 5, 4, 2, 6, 3, 1	Complete	Completion: All 8 graph edges have been traversed exactly once. 

Tucker's Algorithm Execution

Step	Phase	Active Component	Action	State / Result

1	Phase 1: Cycle Decomposition	Unvisited Graph	Start at node 1, traverse unused edges to form the first simple cycle.	Cycle 1 (C1): [1, 2, 4, 5, 3, 1] (Edges: 1-2, 2-4, 4-5, 5-3, 3-1) 
2	Phase 1: Cycle Decomposition	Remaining Edges: (2,3), (2,6), (3,6)	Start at node 2 on remaining unused edges to form the second simple cycle.	Cycle 2 (C2): [2, 6, 3, 2] (Edges: 2-6, 6-3, 3-2)

				All 8 edges partitioned.
3	Phase 2: Cycle Splicing	Base Tour (T) & C2	Set base tour T = C1. Identify shared vertices between T and C2 (vertices 2 and 3 are shared).	Shared nodes: {2, 3}
4	Phase 2: Cycle Splicing	Vertex 3 of tour T	Splice C2 oriented from node 3 (3-6-2-3) by substituting vertex 3 in T.	Final Eulerian Circuit: 1 2 4 5 3 6 2 3 1

B. Comparative Evaluation

A rigorous asymptotic and empirical performance comparison was conducted across all three algorithms:

Algorithm	Time Complexity	Space Complexity	CSES 1691 Feasibility	Key Strengths & Limitations
Hierholzer	$O(V + E)$ [Optimal Linear]	$O(V + E)$	PASS (All Tests) < 0.05s on max test	Optimal: Single-pass traversal, zero bridge checks, minimal memory footprint, very cache-friendly.

Tucker	$O(V + E)$ [Linear]	$O(V + E)$	PASS (All Tests) < 0.06s on max test	Highly Scalable: Clear separation of concerns (cycle discovery + merging). Slightly higher constant factor than Hierholzer due to cycle data structures.
Fleury	$O(E^2)$ [Quadratic]	$O(V + E)$	TLE on Large Tests Time limit 1.0s exceeded	Pedagogical: Intuitive step-by-step logic, but repeated bridge detection via BFS/DFS creates quadratic slowdown on dense/large graphs.